

# Train Ticket Vending Machine

Student: Decean Cosmina Andreea  
Technical University of Cluj-Napoca: Computer Science  
Group: 30415

Year I: Digital System Design  
Lecturer: Miclea Vlad  
Laboratory: Virna Ioana

## TABLE OF CONTENTS

|                                                                                               |          |
|-----------------------------------------------------------------------------------------------|----------|
| <b>1. PROJECT SPECIFICATIONS .....</b>                                                        | <b>3</b> |
| 1.1. OPERATING PROCEDURES.....                                                                | 3        |
| 1.2. CANCEL FEATURE .....                                                                     | 3        |
| 1.3. CONSTRAINTS AND LIMITATIONS .....                                                        | 3        |
| 1.4. USER INTERFACE .....                                                                     | 4        |
| <b>2. DESIGN.....</b>                                                                         | <b>4</b> |
| 2.1. BLACK BOX .....                                                                          | 4        |
| 2.1.1. INPUTS.....                                                                            | 4        |
| 2.1.2. OUTPUTS.....                                                                           | 5        |
| 2.1.3. DIAGRAM.....                                                                           | 5        |
| 2.2. CONTROL AND EXECUTION UNIT .....                                                         | 6        |
| 2.2.1. MAPPING THE INPUTS AND OUTPUTS OF THE BIG BOX AND THE TWO COMPONENTS UC AND UE 6 ..... | 6        |
| 2.2.2. ESTABLISHING THE RESOURCES (EXECUTION UNIT).....                                       | 7        |
| 2.2.3. BLOCK DIAGRAM FOR FIRST BREAKDOWN.....                                                 | 10       |
| 2.3. STATE DIAGRAM OF CONTROL UNIT (UC).....                                                  | 11       |
| 2.3.1. STATE DESCRIPTION.....                                                                 | 11       |
| 2.3.2. THE DIAGRAM.....                                                                       | 12       |
| 2.4. DETAILED DIAGRAM.....                                                                    | 13       |

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>3. DESIGN SOLUTIONS AND JUSTIFICATION .....</b>  | <b>13</b> |
| 3.1. DEBOUNCER .....                                | 13        |
| 3.2. BANKNOTE INVENTORY PACKAGE .....               | 14        |
| 3.3. DISTANCE AND PRICE UNIT .....                  | 15        |
| 3.4. PAYMENT AND LIMIT UNIT .....                   | 15        |
| 3.5. TICKET INVENTORY UNIT.....                     | 17        |
| 3.6. CHANGE AND REFUND UNIT .....                   | 18        |
| 3.7. EXECUTION UNIT .....                           | 20        |
| 3.8. CONTROL UNIT .....                             | 21        |
| 3.9. BCD CONVERTER .....                            | 24        |
| 3.10. HEX TO 7-SEGMENT .....                        | 25        |
| 3.11. TOP MODULE .....                              | 26        |
| <b>4. USER MANUAL .....</b>                         | <b>27</b> |
| <b>5. FURTHER DEVELOPMENT AND IMPROVEMENTS.....</b> | <b>31</b> |
| 5.1. ADVANCED PAYMENT AND BILLING SYSTEM.....       | 31        |
| 5.2. USER INTERFACE ENHANCEMENTS .....              | 31        |
| 5.3. SYSTEM OPTIMISATION .....                      | 31        |
| 5.4. MAINTENANCE MONITORING.....                    | 31        |
| <b>6. BIBLIOGRAPHY .....</b>                        | <b>32</b> |
| 6.1. REFERENCES .....                               | 32        |
| 6.2. WEB AND VIDEO RESOURCES .....                  | 32        |

# 1. PROJECT SPECIFICATIONS

The objective of this project is to design and implement a digital logic control system simulating a train ticket vending machine using a Finite State Machine (FSM) architecture. This system allows the user to input a distance (in tens of kilometers), calculates the price of the train ticket for the respective distance, and accepts payment made in various banknotes and coins (1-100 EURO). The machine provides the exact change, if possible, and dispenses the ticket, if any. If any of the previous actions are not possible, it will show a light signal and refund the money inserted. The machine manages an internal inventory of banknotes and tickets.

## 1.1. OPERATING PROCEDURES

- DISTANCE AND PRICE CALCULATION
  - The machine must accept distances ranging from 10km to 990km.
  - The user enters the distance using two sets of 4-bit BCD switches (input ranges from 1 to 99, representing tens of kilometers).
  - The ticket price must be calculated automatically at a rate of 0.2 EURO per kilometer.
- PAYMENT PROCESSING
  - The machine must accept 1, 2, 5, 10, 20, 50, and 100 EURO coins and banknotes.
  - The machine must display the calculated price on the 7-segment display and update the display accordingly to show the remaining amount to be paid until the required amount is reached.
- TRANSACTION VERIFICATION
  - The machine must verify if the sufficient amount has been inserted and if there are available tickets to dispense.
  - The machine must calculate the exact change and dispense it if the user overpays.
  - The user must be able to cancel any operation at any time and be refunded the full amount inserted.

## 1.2. CANCEL FEATURE

- The vending machine has a button called `cancel_btn`, which, when pressed, automatically cancels everything, refunds all the money inserted if any, and goes back to the IDLE. It can be activated at any time besides the moment when the change is already being dispensed.

## 1.3. CONSTRAINTS AND LIMITATIONS

- TICKET AND BANKNOTE INVENTORY: The machine is initialized with 10 tickets and 10 banknotes, and coins of each type. The maximum capacity for any banknote or coin is 100.

- **TRANSACTION LIMITS:** A maximum of 15 banknotes or coins of a single type can be inserted per transaction.
- **CHANGE LIMITATIONS:** If the machine cannot provide the exact change, then the transaction fails, all operations are canceled, and the inserted money is refunded.

#### 1.4. USER INTERFACE

- **INPUTS:**
  - Switches for inputting distance, banknote types, and system reset (`distance_sw`, `banknote_sw`, `reset_sw`).
  - Buttons for `next_btn`, `cancel_btn`, `insert_btn` and `validate_btn`.
- **DISPLAY:**
  - 7-segment display for showing the distance input, total ticket price, remaining payment, and extra payment to the user (`an`, `cat`).
- **STATUS INDICATORS:**
  - Green LED for successful transaction (`green_led`).
  - Blue LED for inability to give change (`blue_led`).
  - Red LED for unsuccessful transaction or lack of tickets (`red_led`).
  - LED to signal the ticket, change, and refund dispense (`dispense_money_led`, `dispense_ticked_led`).

## 2. DESIGN

### 2.1. BLACK BOX

The Black Box design defines all the external inputs and outputs of the entire top-level system module, while all the implementation details are hidden inside this model.

#### 2.1.1. INPUTS

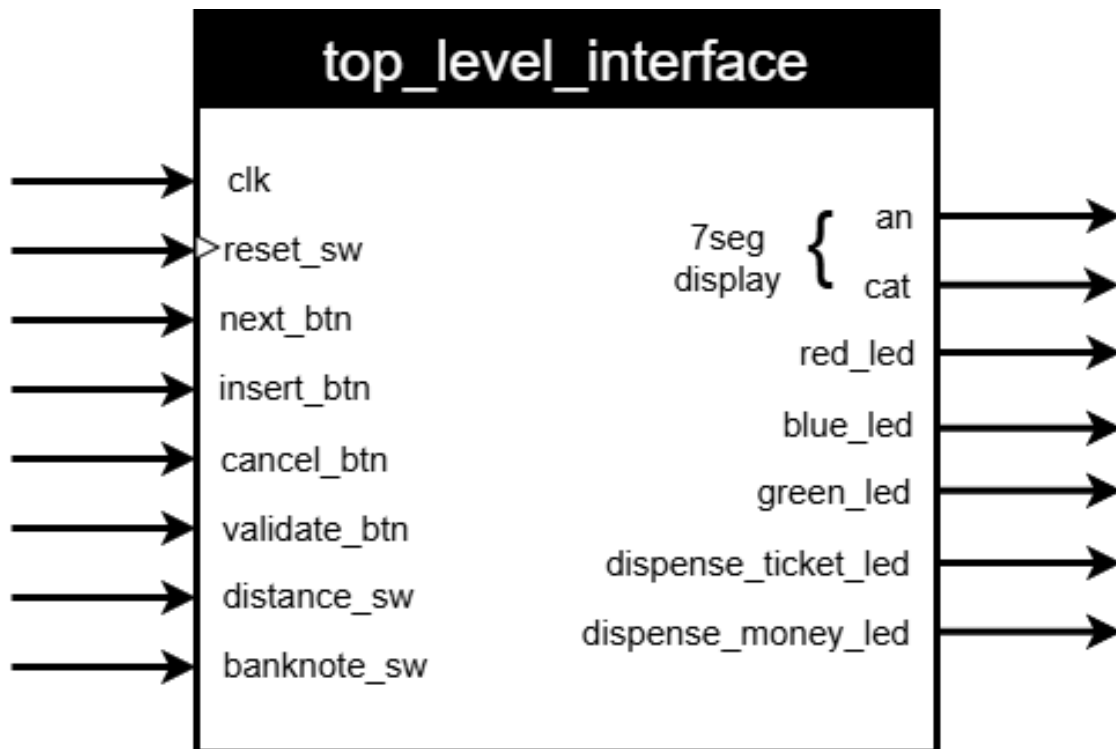
- `clk`: synchronous clock used for various components;
- `reset_sw`: asynchronous active-low reset button used to revert the machine back to the initial settings;
- `next_btn`: button used as confirmation for going from one state to the next one when needed;
- `insert_btn`: button used as confirmation for banknote insertion;

- **cancel\_btn**: button used to cancel every operation, refund the money inserted if needed, and force the machine into IDLE, without resetting the ticket count or the banknote inventory;
- **validate\_btn**: this button can be activated in the VERIFY state, and it shows the final state of the machine after all the user input has been validated;
- **distance\_sw [7:0]**: switches used to input the number of kilometers in BCD format;
- **banknote\_sw [2:0]**: switches used as selection for the banknote being inserted (001 - 1 EURO, 010 - 2 EURO, 011 - 5 EURO, 100 - 10 EURO, 101 - 20 EURO, 110 - 50 EURO, 111 - 100 EURO).

### 2.1.2. OUTPUTS

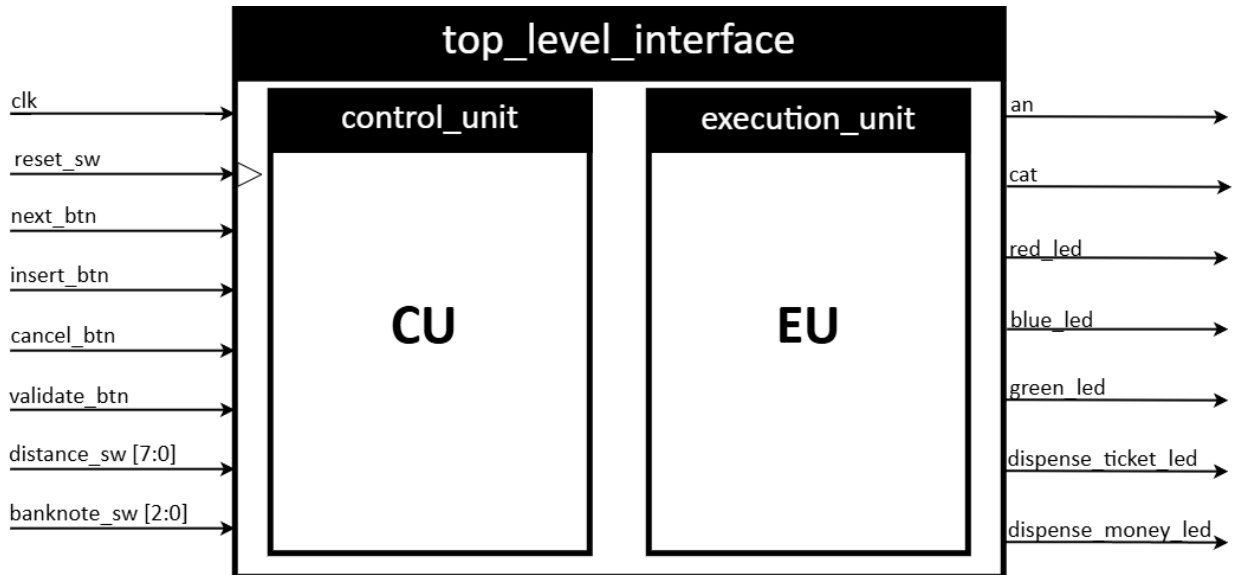
- **an, cat**: 7-segment display anodes and cathodes used for showing the distance input, ticket price, remaining payment, and change;
- **red\_led**: led signaling impossible transaction or lack of tickets;
- **blue\_led**: led signaling impossibility to give change;
- **green\_led**: led signaling a successful transaction;
- **dispense\_ticket\_led**: led signaling ticket dispense;
- **dispense\_money\_led**: led signaling money dispense.

### 2.1.3. DIAGRAM



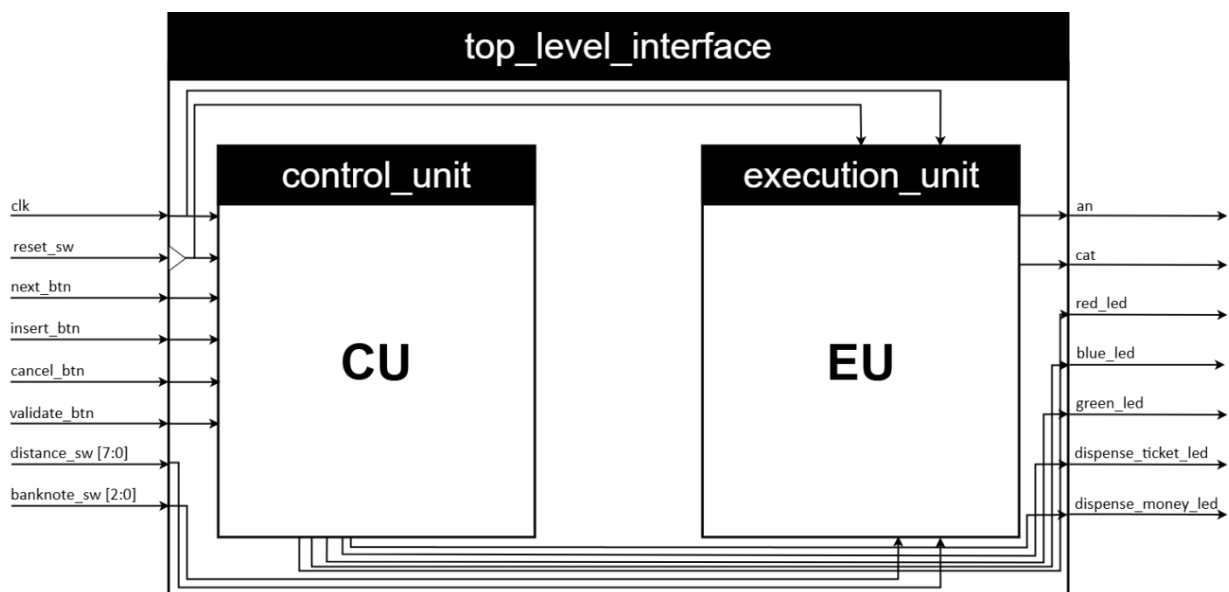
## 2.2. CONTROL AND EXECUTION UNIT

The system's black box must be further decomposed in order to find implementable components. Next, we will do a top-down breakdown of the problem until we get to known circuits, and then we will implement bottom-up.



In this first breakdown of the system, we can differentiate between the control logic in the system and the system resources. The control logic is represented by the Control Unit (CU), and the resources are represented by the Execution Unit (EU).

### 2.2.1. MAPPING THE INPUTS AND OUTPUTS OF THE BIG BOX AND THE TWO COMPONENTS UC AND UE



We can divide both inputs and outputs into 2 categories: data and control. This separation is essential in the beginning so we can simplify our implementation later on.

- DATA INPUTS: values for different data (distance, ticket cost, number of tickets, value of change, number of banknotes, type of banknote, payment value)
- CONTROL INPUTS: confirmations, resets, selections (next button, cancel button, reset button, money insert confirmation button)
- DATA OUTPUTS: values displayed to the user (ticket cost, number of kilometers, remaining payment, change)
- CONTROL OUTPUTS: warnings, and signals that guide the user (LEDs)

### 2.2.2. ESTABLISHING THE RESOURCES (EXECUTION UNIT)

In order to further establish the links between the CU and EU, we must identify the resources on the basis of which we make decisions. These resources generate signals to facilitate the communication between the CU and EU. These signals are of 2 types: control commands (from the CU to the EU) and status flags (from the EU to the CU).

#### CONTROL COMMANDS

- **load\_distance**: when **next\_btn** is pressed after the distance is inputted on the switches, it generates this signal to tell the EU to load the distance into a register and then calculates the ticket price
- **add\_money**: when **insert\_btn** is pressed, it generates a signal that tells the EU to add the money to the **inserted\_amount** and subtracts from the **remaining\_payment** register, and increments the number of banknotes of the type inserted
- **decrement\_tickets**: if the transaction is successful, tells the EU to decrement the ticket count permanently.
- **dispense\_trigger**: trigger for the banknote dispense for either change or refund
- **clear\_all**: resets the transaction registers if cancel is pressed and triggers the refund if needed

#### STATUS FLAGS

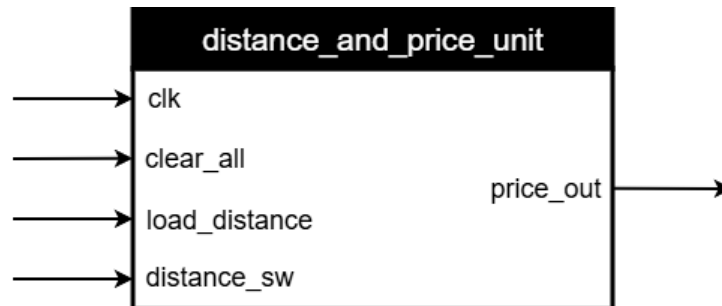
- **is enough**: after **next\_btn** is pressed and the wanted amount of money is inserted, the machine compares **inserted\_amount** to price and generates a signal to tell the CU that the amount of money inserted is enough for the ticket

- no\_tickets: if the tickets = 0, it generates a signal to tell the CU there are no tickets left.
- banknote\_limit: triggers when any type of banknote inserted is greater than 15
- change\_possible: signal to indicate that the current banknote inventory can fulfill the change requirement
- valut\_full: triggers when the internal inventory of any banknote type reaches the 100 limit.

## RESOURCES

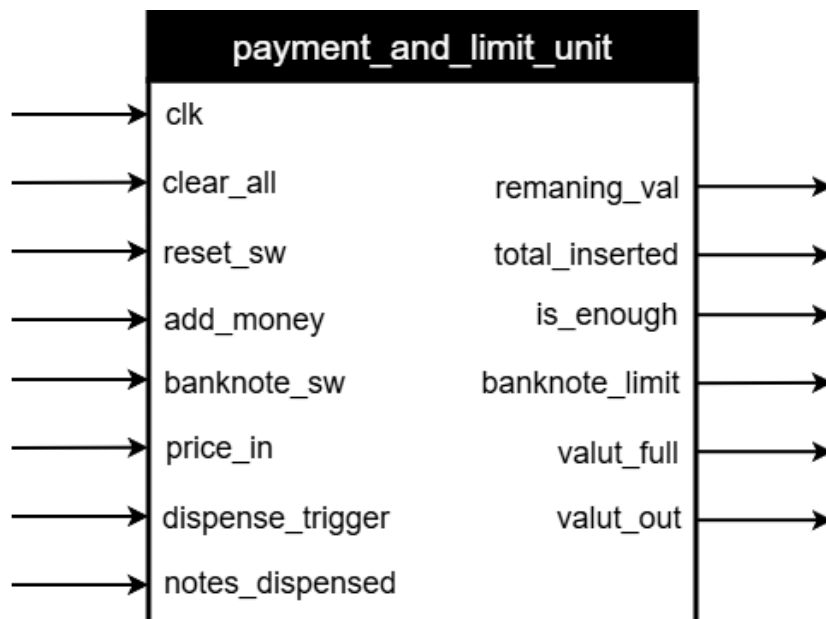
### 1. Distance and price unit

This module is composed of a register and other logic components, and it performs a distance \* 2 operation to calculate the ticket price. It's reset upon the reset signal.



### 2. Payment and limit unit

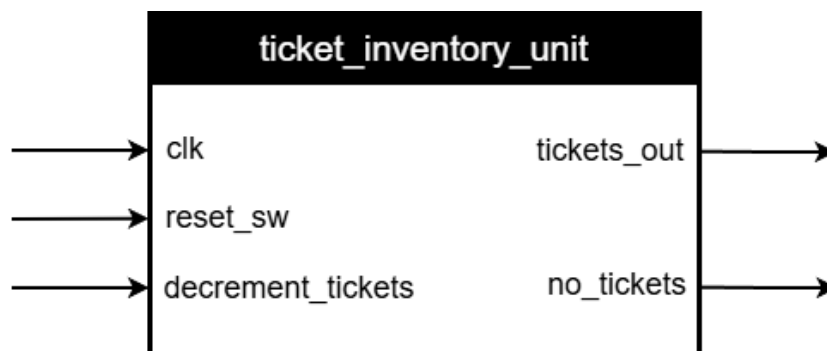
This module is composed of a multiplexer for selecting the banknote inserted, registers, adder-subtractors, and has multiple outputs such as the remaining payment, whether the total payment is enough for a ticket, if the banknote count reaches the limit, and if the valut inventory is full. It is reset upon the reset signal.





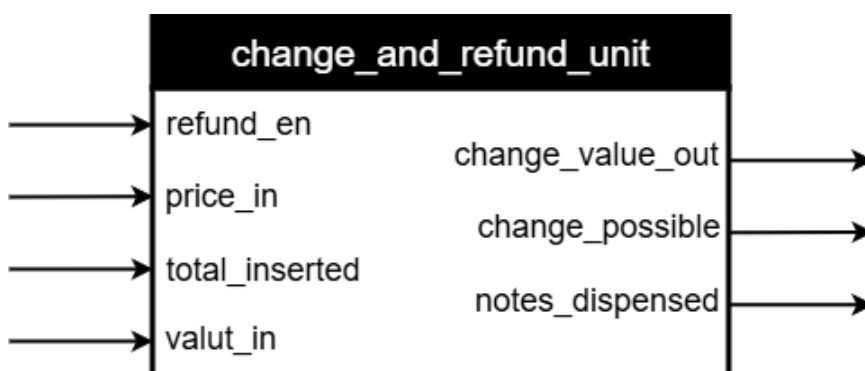
### 3. Ticket inventory unit

This module is composed of multiple registers that check if there are available tickets and decrement them. The initial number of tickets is 10 and will be reseted to this value upon the reset signal.



### 4. Change and refund unit

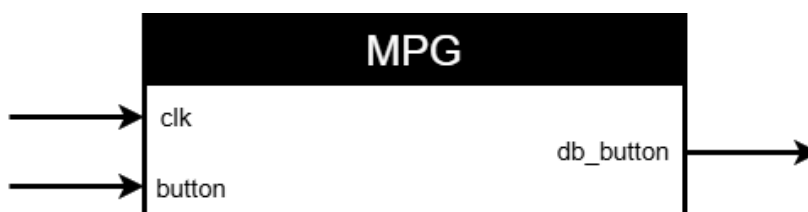
This module is composed of registers, comparators, and takes input from the price, total paid, and the data from the valut unit, and it outputs signals for the possibility to give change to the user and its value.



## 5. Components outside the EU and CU

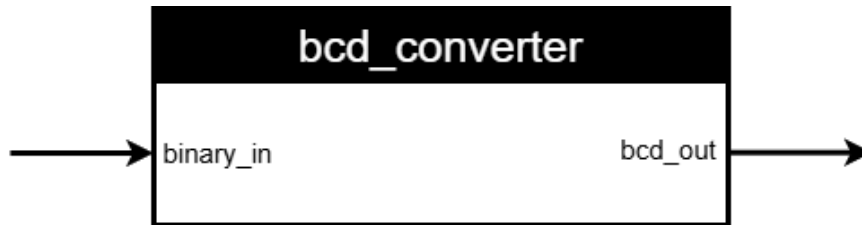
### 5.1. Debouncer (MPG)

This is provided in the laboratory resources; essentially, it makes the button presses clean, and the machine is able to register them.



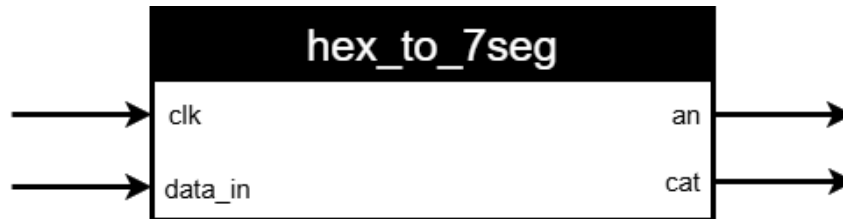
### 5.2. Binary to BCD converter

This component decodes a binary number into a BCD-packed number, essential for displaying the price, remaining payment, and change to the user.



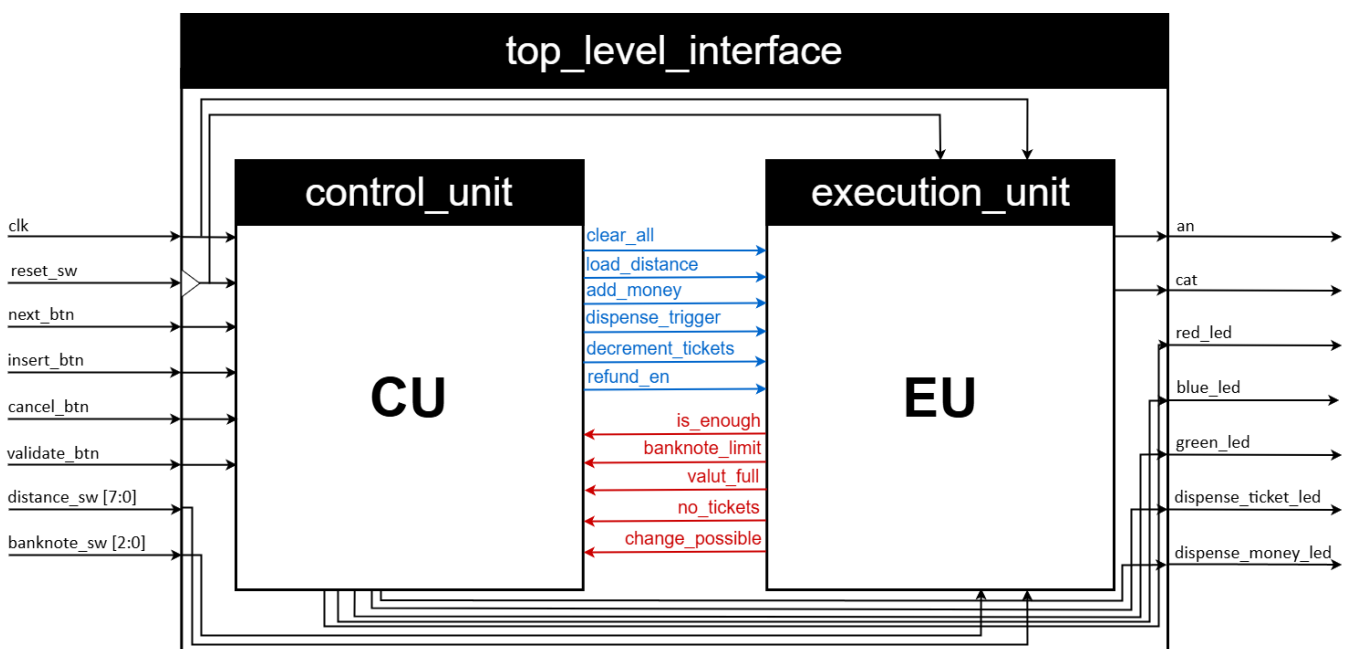
### 5.3. Hex to 7-segment display controller

While originally provided in the laboratory, I added a frequency divider to implement the refresh rate of the display.



#### 2.2.3. BLOCK DIAGRAM FOR FIRST BREAKDOWN

Having these components, we can now visualize the communication paths between the Control Unit and the Execution Unit.



## 2.3. STATE DIAGRAM OF CONTROL UNIT (UC)

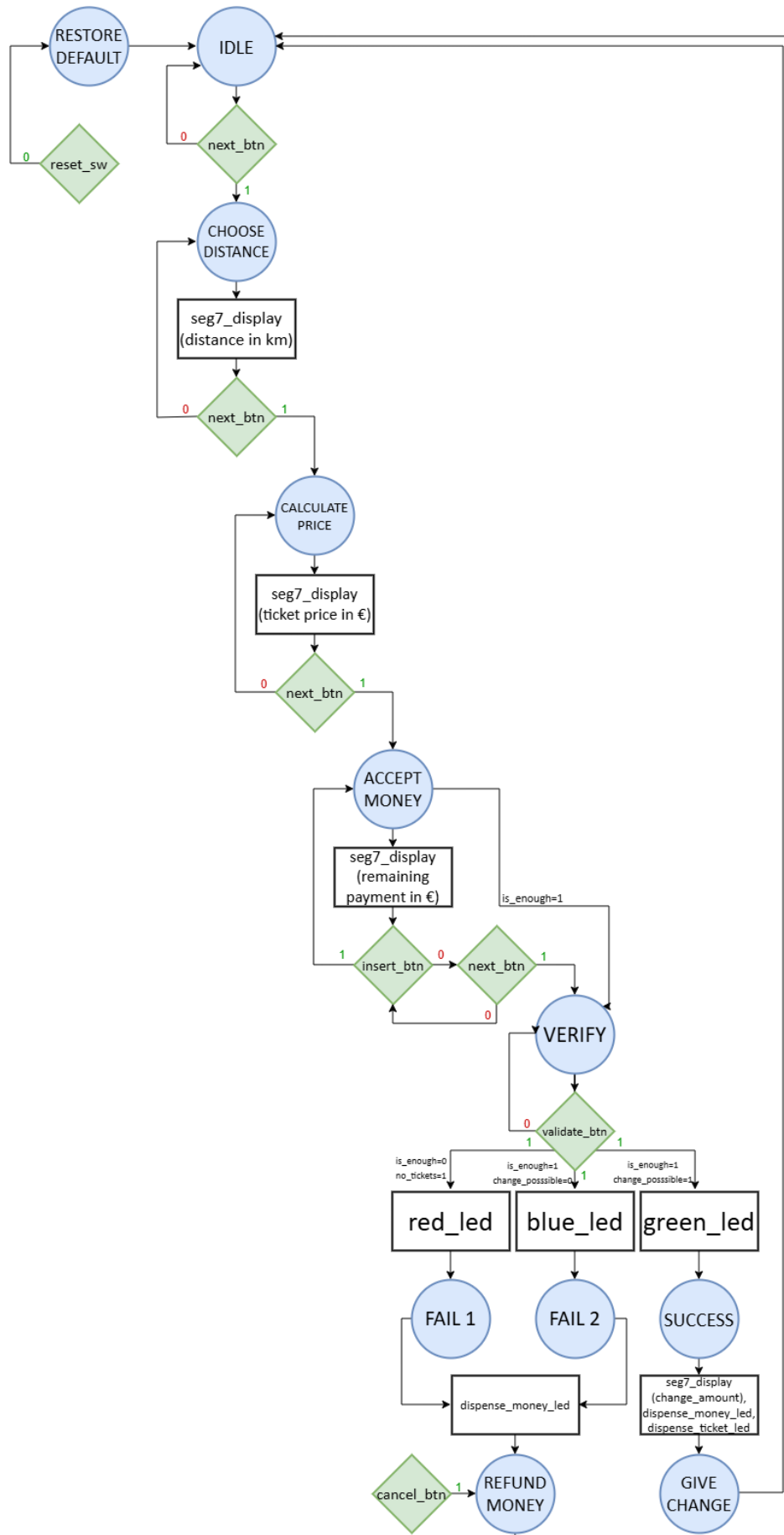
The state diagram represents the decision part of the machine algorithm. It is composed of states, decisions, and outputs.

- STATES are represented by a circle, and they stand for a moment in time.
- The DECISIONS are represented by a rhombus. These are inputs taken from the user.
- The OUTPUTS are represented by a rectangle. If they are before the decision, it means that the outputs depend only on the internal state, so we have a MOORE automation. If they are after the decision, it means that the output also depends on the internal state and the inputs, so we have a MEALY automation.

### 2.3.1. STATE DESCRIPTION

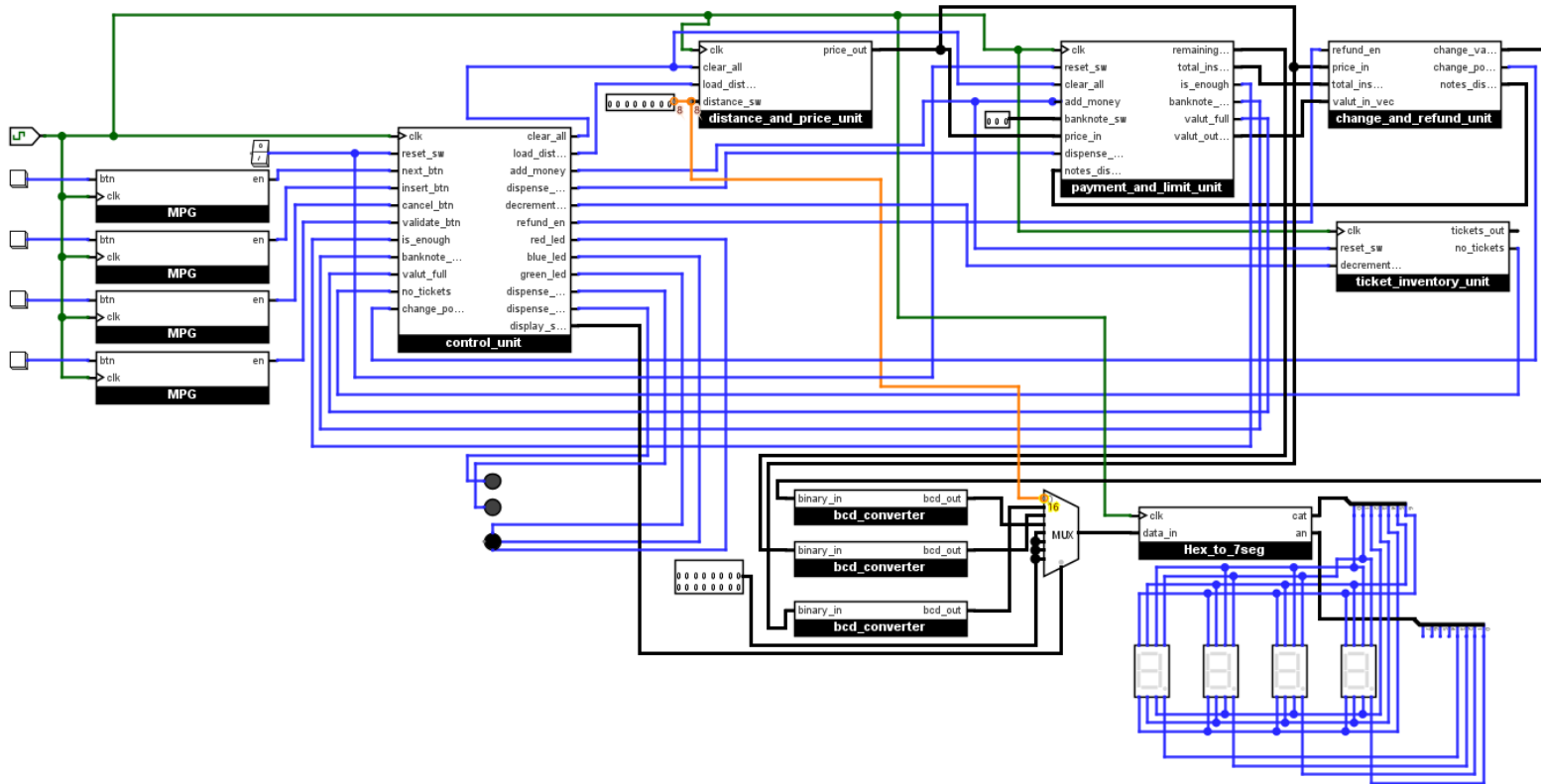
- IDLE: The vending machine is in idle mode and is waiting for input
- RESTORE\_DEFAULT: By switching off `reset_sw`, restores the ticket and banknote inventory to default (10 of each piece)
- CHOOSE\_DISTANCE: Waits for the user to use the switches to choose a custom distance. Waits for `next_btn = 1`.
- SHOW\_PRICE: Displays the ticket price on the 7-segment display. Waits for `next_btn = 1`.
- ACCEPT\_MONEY: The user chooses the banknote to be inserted using the switches, waits for `insert_btn = 1` to confirm the inserted banknote, and performs subtraction on `price`. When `next_btn = 1`, it exists the state. If `inserted_amount >= price`, automatically exists the state.
- VERIFY: This state has 3 different branches after `validate_btn` is pressed:
  - If `is_enough = 0` (`inserted_amount < price`) OR `no_tickets = 1`, it goes to FAIL\_1 (Red LED).
  - If `is_enough = 1` (`inserted_amount >= price`) AND `change_possible = 0`, it goes to FAIL\_2 (Blue LED).
  - If `is_enough = 1` (`inserted_amount >= price`) AND `change_possible = 1`, it goes to SUCCESS (Green LED).
- SUCCESS: Turns on `green_led`. Dispenses the ticket, gives the change, both are signaled by LEDs, Goes to IDLE.
- FAIL\_1: Turns on the `red_led`. Goes to REFUND state.
- FAIL\_2: Turns on the `blue_led`. Goes to REFUND state.
- REFUND: Can also be triggered by `cancel_btn = 1`. Refunds `inserted_amount` and is signaled by an LED. Goes to IDLE.

### 2.3.2. THE DIAGRAM



## 2.4. DETAILED DIAGRAM

The detailed diagram has been realized in Logisim Evolution and represents how each of the components are connected.



## 3. DESIGN SOLUTIONS AND JUSTIFICATION

### 3.1. DEBOUNCER

This unit was provided in the laboratory, and it essentially consolidates multiple rapid, noisy, and frequent inputs into a single intentional action, making the button press be recognized for the whole duration, not just for a single clock pulse.

architecture Behavioral of MPG is

```

signal count      : STD_LOGIC_VECTOR (15 downto 0) := (others => '0');
signal t          : STD_LOGIC;
signal q1         : STD_LOGIC;
signal q2         : STD_LOGIC;
signal q3         : STD_LOGIC;

```

```
begin

    process (clk)
    begin

        if (rising_edge(clk)) then
            count <= count + 1;
        end if;

    end process;

    t <= '1' when count = x"FFFF" else '0';

    process(clk)
    begin

        if (rising_edge(clk)) then
            if(t = '1') then
                q1 <= btn;
            end if;
        end if;

    end process;

    process(clk)
    begin

        if (rising_edge(clk)) then
            q2 <= q1;
            q3 <= q2;
        end if;

    end process;

    en <= q2 and not q3;

end Behavioral;
```

---

### 3.2. BANKNOTE INVENTORY PACKAGE

This module defines a shared package. It establishes a custom array type called `valut_array`, which is configured as an array from 1 to 7 to store integers ranging from 0 to 127. This allows the same array structure to be utilized across both the payment unit and the change unit to track the banknote inventory.

---

```
package banknote_inventory_package is

    type valut_array is array (1 to 7) of integer range 0 to 127;

end package banknote_inventory_package;
```

---

### 3.3. DISTANCE AND PRICE UNIT

This module is responsible for interpreting the user's destination distance and computing the respective ticket cost. It converts the 8-bit BCD user distance switch input into an integer representing the total distance. Because the input is assumed to be in tens of kilometers, it simply multiplies the total distance by 2 to obtain the final internal 16-bit ticket price.

---

architecture Behavioral of distance\_and\_price\_unit is

```
signal internal_price : unsigned (15 downto 0) := (others => '0');
```

```
begin
```

```
process (clk)
```

```
variable first_4_digits : integer range 0 to 9;  
variable last_4_digits : integer range 0 to 9;  
variable total_distance : integer range 0 to 127;
```

```
begin
```

```
--clear signal to reset the registers
```

```
if rising_edge(clk) then
```

```
if clear_all = '1' then
```

```
internal_price <= (others => '0');
```

```
elsif load_distance = '1' then
```

```
--converts bcd to integer
```

```
first_4_digits := to_integer(unsigned(distance_sw(7 downto 4)));
```

```
last_4_digits := to_integer(unsigned(distance_sw(3 downto 0)));
```

```
--calculate the actual distance
```

```
total_distance := (first_4_digits * 10) + last_4_digits;
```

```
internal_price <= to_unsigned(total_distance * 2, 16); -- because the distance is in tens  
of km, we don't have to multiply by 0.2, only by 2
```

```
end if;
```

```
end if;
```

```
end process;
```

```
price_out <= std_logic_vector(internal_price);
```

```
end Behavioral;
```

---

### 3.4. PAYMENT AND LIMIT UNIT

This unit processes the input currency while monitoring and enforcing mechanical safety limits. It identifies the inserted banknote value via switches and adds it to the inserted\_sum. It initializes with a permanent vault inventory of 10 banknotes of each type. It prevents the





```

    elsif dispense_trigger = '1' then
        --subtract what banknotes we need to use to give the change (calculated in the change
        unit)
        for i in 1 to 7 loop
            valut_inventory(i) <= valut_inventory(i) - notes_dispensed(i);
        end loop;
    end if;
end if;
end process;

banknote_limit <= '1' when (
    (transaction_count(1) >= 15) or
    (transaction_count(2) >= 15) or
    (transaction_count(3) >= 15) or
    (transaction_count(4) >= 15) or
    (transaction_count(5) >= 15) or
    (transaction_count(6) >= 15) or
    (transaction_count(7) >= 15))
else '0';

valut_full <= '1' when (
    (valut_inventory(1) >= 100) or
    (valut_inventory(2) >= 100) or
    (valut_inventory(3) >= 100) or
    (valut_inventory(4) >= 100) or
    (valut_inventory(5) >= 100) or
    (valut_inventory(6) >= 100) or
    (valut_inventory(7) >= 100))
else '0';

total_inserted <= std_logic_vector(inserted_sum);

valut_out    <= valut_inventory; -- share valut data with the change unit

is_enough <= '1' when
    inserted_sum >= unsigned(price_in)
else '0';

remaining_val <= std_logic_vector(unsigned(price_in) - inserted_sum) when
    unsigned(price_in) > inserted_sum
else (others => '0');

end Behavioral;

```

---

### 3.5. TICKET INVENTORY UNIT

A simple counter module dedicated to tracking the physical ticket stock inside the machine. The machine initializes with a count of 10 tickets. Every time a ticket is dispensed, the unit decrements the count. If the inventory falls to 0, it asserts the `no_tickets` output flag.

---

architecture Behavioral of ticket\_inventory\_unit is

```

    signal ticket_count : unsigned(3 downto 0) := to_unsigned(10, 4); --initialize at 10 tickets

```

```

begin

    process (clk)
    begin

        if rising_edge(clk) then
            if reset_sw = '0' then
                ticket_count <= to_unsigned(10, 4); -- restore default
            elsif decrement_tickets = '1' then
                if ticket_count > 0 then
                    ticket_count <= ticket_count - 1;
                end if;
            end if;
        end if;
    end process;

    tickets_out <= std_logic_vector(ticket_count);
    --checks for no tickets
    no_tickets <= '1' when
        ticket_count = 0
    else '0';

end Behavioral;

```

---

### 3.6. CHANGE AND REFUND UNIT

This unit manages the mathematics of returning money to the user through a Greedy Algorithm. Depending on the `refund_en` signal, it either targets a full refund of the inserted cash or calculates the required change by subtracting the price from the total inserted. It employs a Greedy Algorithm to determine exactly how many 100, 50, 20, 10, 5, 2, and 1 euro banknotes or coins must be dispensed based on the current `valut` inventory. If the exact change can be made, it asserts the `change_possible` flag.

---

architecture Behavioral of change\_and\_refund\_unit is

```

begin

    process (refund_en, price_in, total_inserted, valut_in)

        variable target          : integer;
        variable remaining       : integer;
        variable needed          : integer;
        variable to_give         : integer;
        variable valut_data      : valut_array;
        variable notes_used      : valut_array;

    begin

        if refund_en = '1' then
            target := to_integer(unsigned(total_inserted));

```

```
else
  if to_integer(unsigned(total_inserted)) >= to_integer(unsigned(price_in)) then
    target := to_integer(unsigned(total_inserted)) - to_integer(unsigned(price_in));
  else
    target := 0; -- negative change safety
  end if;
end if;

remaining := target;
valut_data := valut_in;
notes_used := (others => 0);
change_value_out <= std_logic_vector(to_unsigned(target, 16));
change_possible <= '0';
notes_dispensed <= (others => 0);

-- greedy algorithm to calculate how many banknotes of each type to give
if target > 0 then

  -- 100 euro (index 7)
  needed := remaining / 100;
  if needed > valut_data(7) then
    to_give := valut_data(7);
  else
    to_give := needed;
  end if;
  remaining := remaining - (to_give * 100);
  notes_used(7) := to_give;

  -- 50 euro (index 6)
  needed := remaining / 50;
  if needed > valut_data(6) then
    to_give := valut_data(6);
  else
    to_give := needed;
  end if;
  remaining := remaining - (to_give * 50);
  notes_used(6) := to_give;

  -- 20 euro (index 5)
  needed := remaining / 20;
  if needed > valut_data(5) then
    to_give := valut_data(5);
  else
    to_give := needed;
  end if;
  remaining := remaining - (to_give * 20);
  notes_used(5) := to_give;

  -- 10 euro (index 4)
  needed := remaining / 10;
  if needed > valut_data(4) then
    to_give := valut_data(4);
  else
    to_give := needed;
  end if;
  remaining := remaining - (to_give * 10);
  notes_used(4) := to_give;
```

```

-- 5 euro (index 3)
needed := remaining / 5;
if needed > valut_data(3) then
    to_give := valut_data(3);
else
    to_give := needed;
end if;
remaining := remaining - (to_give * 5);
notes_used(3) := to_give;

-- 2 euro (index 2)
needed := remaining / 2;
if needed > valut_data(2) then
    to_give := valut_data(2);
else
    to_give := needed;
end if;
remaining := remaining - (to_give * 2);
notes_used(2) := to_give;

-- 1 euro (index 1)
needed := remaining / 1;
if needed > valut_data(1) then
    to_give := valut_data(1);
else
    to_give := needed;
end if;
remaining := remaining - (to_give * 1);
notes_used(1) := to_give;

--check if change is possible
if remaining = 0 then
    change_possible <= '1';
    notes_dispensed <= notes_used;
else
    change_possible <= '0';
    notes_dispensed <= (others => 0);
end if;

elsif target = 0 then
    change_possible <= '1';
    notes_dispensed <= (others => 0);
end if;

end process;

end Behavioral;

```

---

### 3.7. EXECUTION UNIT

This is a structural datapath module that acts as a central hub. It groups the various functional sub-components together, including the distance\_and\_price\_unit, payment\_and\_limit\_unit,

ticket\_inventory\_unit, and change\_and\_refund\_unit. It bridges the communication between these units and the control unit by receiving command signals and transmitting status flags.

---

```

architecture Structural of execution_unit is
  --component declarations
  component distance_and_price_unit is
  end component distance_and_price_unit;

  component payment_and_limit_unit is
  end component payment_and_limit_unit;

  component ticket_inventory_unit is
  end component ticket_inventory_unit;

  component change_and_refund_unit is
  end component change_and_refund_unit;

  --internal signals to communicate between units
  signal sig_price      : STD_LOGIC_VECTOR (15 downto 0);
  signal sig_total_inserted : STD_LOGIC_VECTOR (15 downto 0);
  signal sig_valut_data   : valut_array;
  signal sig_notes_dispensed : valut_array;

begin

  u1_distance: distance_and_price_unit port map
  u2_payment: payment_and_limit_unit port map
  u3_tickets: ticket_inventory_unit port map
  u4_change: change_and_refund_unit port map

  price_out    <= sig_price;
  total_inserted <= sig_total_inserted;

end Structural;

```

---

### 3.8. CONTROL UNIT

This file acts as the “brain” of the vending machine, functioning as a Finite State Machine (FSM). It manages the overall operational sequence with states including IDLE, CHOOSE\_DISTANCE, SHOW\_PRICE, ACCEPT\_MONEY, VERIFY, SUCCESS, FAIL\_1, FAIL\_2, and REFUND. Based on the user inputs and status flags, it generates the necessary control commands to trigger actions (like dispensing items or enabling refunds). It also dictates which information state should be broadcast to the 7-segment display via a multiplexer.

---

```

architecture Behavioral of control_unit is

  type state_type is (IDLE, CHOOSE_DISTANCE, SHOW_PRICE, ACCEPT_MONEY, VERIFY,
  SUCCESS, FAIL_1, FAIL_2, REFUND);
  signal current_state, next_state : state_type;

begin

```

---

```
--reset logic
process (clk)
begin

    if rising_edge(clk) then
        if reset_sw = '0' then
            current_state <= IDLE;
        else
            current_state <= next_state;
        end if;
    end if;

end process;

process (current_state, next_btn, insert_btn, cancel_btn, is_enough, banknote_limit, valut_full,
no_tickets, change_possible)
begin

    clear_all      <= '0';
    load_distance  <= '0';
    add_money      <= '0';
    dispense_trigger <= '0';
    decrement_tickets <= '0';
    refund_en      <= '0';

    red_led        <= '0';
    blue_led       <= '0';
    green_led      <= '0';
    dispense_ticket_led <= '0';
    dispense_money_led <= '0';
    display_state   <= "100";

    next_state <= current_state;

--global cancel feature
if cancel_btn = '1' and ((current_state = CHOOSE_DISTANCE) or (current_state =
SHOW_PRICE) or (current_state = ACCEPT_MONEY) or (current_state = VERIFY)) then
    next_state <= REFUND;
else
    case current_state is
        --transition from idle too choosing the distance
        when IDLE =>
            clear_all <= '1';
            if next_btn = '1' then
                next_state <= CHOOSE_DISTANCE;
            end if;
        --transition from distance to showing the ticket price
        when CHOOSE_DISTANCE =>
            display_state <= "000"; -- tells the display to show the distance
            if next_btn = '1' then
                load_distance <= '1';
                next_state <= SHOW_PRICE;
            end if;
        --transition from showing the price to the transaction process
        when SHOW_PRICE =>
            display_state <= "001"; --tells the displays to show the ticket price
            if next_btn = '1' then
```

```

        next_state <= ACCEPT_MONEY;
    end if;
--transition from the transaction process to the verification process
when ACCEPT_MONEY =>
    display_state <= "010"; --tells the display to show the remaining payment amount
    if insert_btn = '1' then
        add_money <= '1';
    end if;
    if (next_btn = '1') or (is_enough = '1') then
        next_state <= VERIFY;
    end if;
--checking for failure or success
when VERIFY =>
    display_state <= "010";
    if validate_btn = '1' then
        if (is_enough = '0') or (no_tickets = '1') then
            next_state <= FAIL_1;
        elsif change_possible = '0' then
            next_state <= FAIL_2;
        else
            next_state <= SUCCESS;
        end if;
    end if;
--dispensing change and tickets if successful
when SUCCESS =>
    display_state <= "011"; -- tells the display to show the change
    green_led <= '1';
    dispense_ticket_led <= '1';
    dispense_money_led <= '1';
    if next_btn = '1' then
        decrement_tickets <= '1';
        dispense_trigger <= '1';
        next_state <= IDLE;
    end if;
--giving refund if fail
when FAIL_1 =>
    red_led <= '1';
    if next_btn = '1' then
        next_state <= REFUND;
    end if;

when FAIL_2 =>
    blue_led <= '1';
    if next_btn = '1' then
        next_state <= REFUND;
    end if;
--refunding the inserted money
when REFUND =>
    dispense_money_led <= '1';
    refund_en <= '1';
--display state is default 100 now
    if next_btn = '1' then
        dispense_trigger <= '1';
        next_state <= IDLE;
    end if;

when others =>

```

```

        next_state <= IDLE;

    end case;
end if;

end process;

end Behavioral;

```

---

### 3.9. BCD CONVERTER

This component translates binary values into a format suitable for the 7-segment display. It takes a 16-bit binary input and converts it into a 16-bit BCD output. It achieves this using the “shift and add 3” algorithm within a 16 iteration loop. If a BCD digit exceeds 4 during the process, it adds 3 before performing a left bit shift.

---

```

architecture Behavioral of bcd_converter is
begin

    process (binary_in)

        variable temp_binary  : unsigned (15 downto 0);
        variable bcd          : unsigned (15 downto 0);

    begin

        temp_binary := unsigned(binary_in);
        bcd         := (others => '0');

        for i in 0 to 15 loop
            -- if the bcd digit is > 4 then add 3 before we shift
            if bcd(3 downto 0) > 4 then
                bcd(3 downto 0) := bcd(3 downto 0) + 3;
            end if;

            if bcd(7 downto 4) > 4 then
                bcd(7 downto 4) := bcd(7 downto 4) + 3;
            end if;

            if bcd(11 downto 8) > 4 then
                bcd(11 downto 8) := bcd(11 downto 8) + 3;
            end if;

            if bcd(15 downto 12) > 4 then
                bcd(15 downto 12) := bcd(15 downto 12) + 3;
            end if;
            --left shifting
            bcd := bcd(14 downto 0) & temp_binary(15);
            temp_binary := temp_binary(14 downto 0) & '0';

        end loop;
    end process;
end architecture;

```



```

bcd_out <= std_logic_vector(bcd);

end process;

end Behavioral;

```

---

### 3.10. HEX TO 7-SEGMENT

This module was initially provided in the laboratory, but I have modified it, and it acts as the display controller, designed to drive a 4-digit 7-segment multiplexer display on the FPGA board. It implements a 20-bit `refresh_counter` that acts as a frequency divider, allowing the display to refresh continuously at a speed that appears solid to the human eye. By reading the top two bits of this counter, it rapidly cycles through four states to activate one digit's anode (an) at a time using active-low logic. As it cycles, it extracts the corresponding 4-bit segment from the 16-bit `data_in` vector and feeds it through a built-in BCD-to-7 segment decoder. This decoder maps the 4-bit value to the 7-bit cathode output (`cat`), lighting up the correct segments (a – g) to display the numbers 0 – 9.

---

architecture Behavioral of Hex\_to\_7seg is

```

signal refresh_counter      : STD_LOGIC_VECTOR (19 downto 0) := (others => '0');
signal led_activating_counter : STD_LOGIC_VECTOR (1 downto 0);
signal led_bcd              : STD_LOGIC_VECTOR (3 downto 0);

```

```
begin
```

```

--refresh the display
process (clk)
begin

```

```

    if rising_edge(clk) then
        refresh_counter <= refresh_counter + 1;
    end if;

```

```
end process;
```

```
led_activating_counter <= refresh_counter(19 downto 18);
```

```
process (led_activating_counter, data_in)
begin

```

```

    --anode activation
    case led_activating_counter is
        when "00" =>
            an <= "11111110"; -- activate digit 0
            led_bcd <= data_in(3 downto 0);
        when "01" =>
            an <= "11111101"; -- activate digit 1
            led_bcd <= data_in(7 downto 4);

```

```

when "10" =>
    an <= "11111011"; -- activate digit 2
    led_bcd <= data_in(11 downto 8);
when "11" =>
    an <= "11110111"; -- activate digit 3
    led_bcd <= data_in(15 downto 12);
when others =>
    an <= "11111111"; -- turn all off
    led_bcd <= "0000";
end case;

end process;

--decode the cathodes
process(led_bcd)
begin

    case led_bcd is
        when "0000" => cat <= "1000000"; --0
        when "0001" => cat <= "1111001"; --1
        when "0010" => cat <= "0100100"; --2
        when "0011" => cat <= "0110000"; --3
        when "0100" => cat <= "0011001"; --4
        when "0101" => cat <= "0010010"; --5
        when "0110" => cat <= "0000010"; --6
        when "0111" => cat <= "1111000"; --7
        when "1000" => cat <= "0000000"; --8
        when "1001" => cat <= "0010000"; --9
        when others => cat <= "1111111"; --turn off
    end case;

end process;

end Behavioral;

```

---

### 3.11. TOP MODULE

This is the highest-level entity that wraps all logical operations together and maps them to physical board inputs and outputs. It instantiates and wires together the `execution_unit` and `control_unit`. Crucially, it includes the MPG debouncer components for the hardware push-buttons to ensure clean signal inputs. It also utilizes multiple `bcd_converted` instances to format the price, remaining value, and change outputs properly for the `Hex_to_7seg` display controller.

---

architecture Structural of top\_level\_interface is

```

--debounced buttons signals
--internal signals

component MPG is
end component MPG;

```

```

component execution_unit is
end component execution_unit;

component control_unit is
end component control_unit;

component bcd_converter is
end component bcd_converter;

component Hex_to_7seg is
end component Hex_to_7seg;

begin

    debouncer_next: MPG port map
    debouncer_insert: MPG port map
    debouncer_cancel: MPG port map
    debouncer_validate: MPG port map
    eu: execution_unit port map
    cu: control_unit port map

    ext_price    <= price_out;
    ext_remaining <= remaining_val;
    ext_change    <= change_value_out;

    display_price: bcd_converter port map
    display_remaning: bcd_converter port map
    display_change: bcd_converter port map
    display: Hex_to_7seg port map

    --7 segment display multiplexer logic
    process (display_state, distance_sw, price_out, remaining_val, change_value_out)
    begin

        case display_state is
            when "000" => s7d_data <= "0000" & distance_sw & "0000";
            when "001" => s7d_data <= bcd_price;
            when "010" => s7d_data <= bcd_remaining;
            when "011" => s7d_data <= bcd_change;
            when others => s7d_data <= (others => '0');
        end case;

    end process;

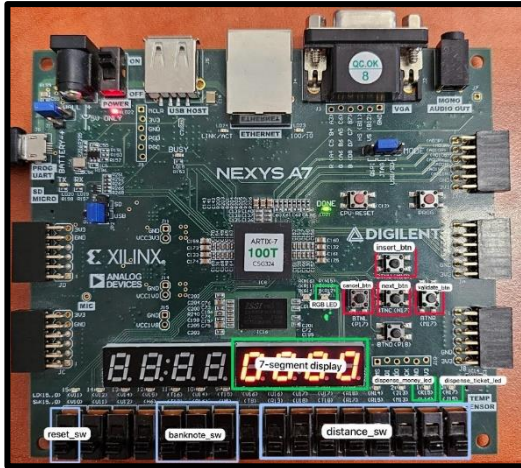
end Structural;

```

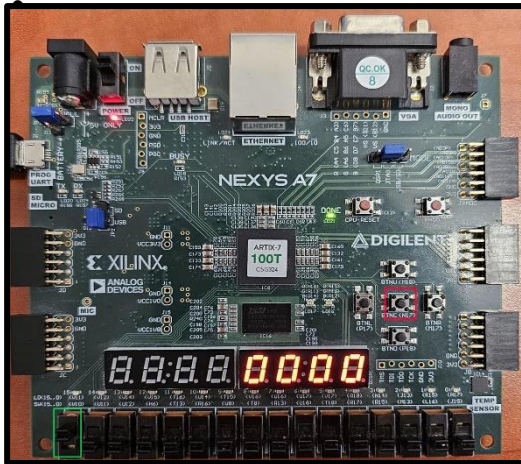
---

## 4. USER MANUAL

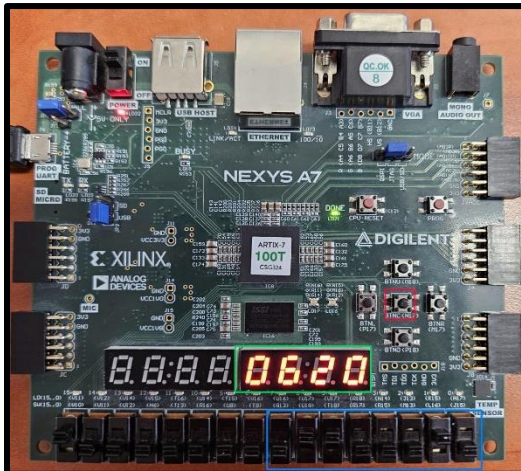
These instructions will be presented through pictures of the FPGA Nexys7 board.



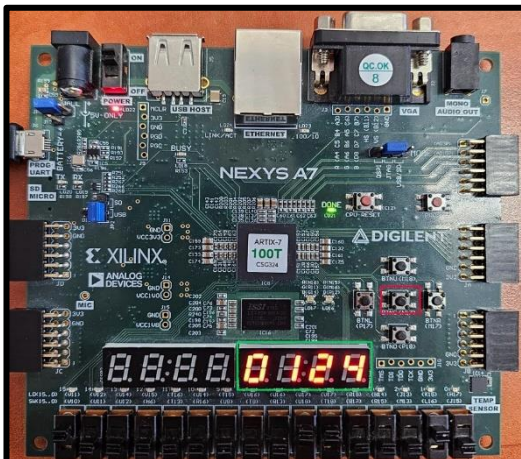
- This represents the IDLE state of the board; you can see this when you first start the program.



- Now you have to flip the reset switch to enable all the functions of the machine; it's like a turn-on button. After that, press next to go into the CHOOSE\_DISTANCE state.

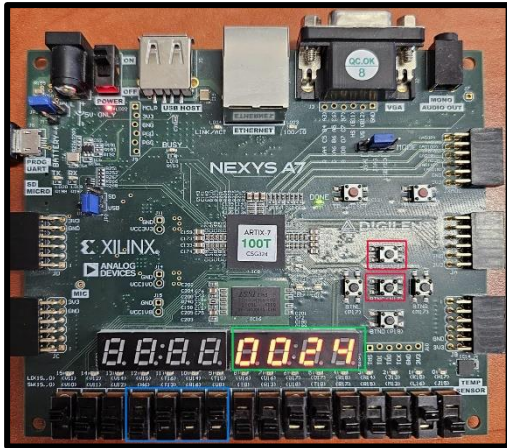


- Use the switches to input the distance you want to travel in BCD format in tens of kilometers. The display will update as you flip the switches. Press next to go into SHOW\_PRICE state.

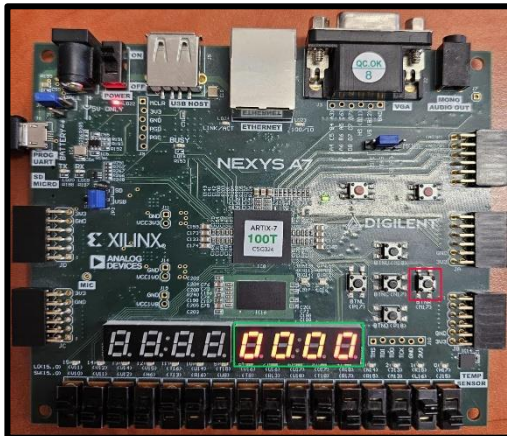


- The ticket price will be displayed on the 7-segment display. Press next to go into ACCEPT\_MONEY state.

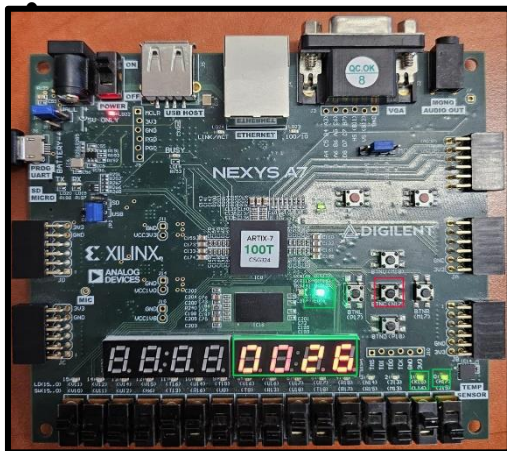




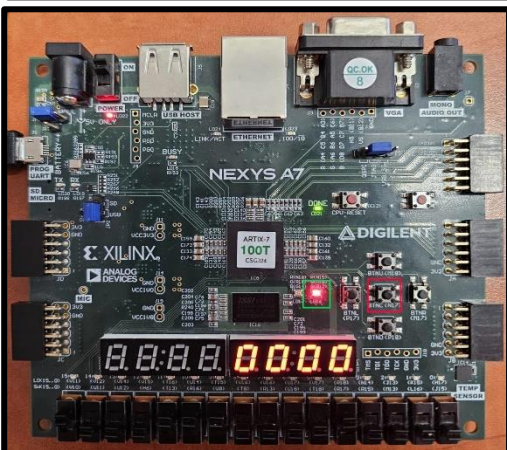
- Having the price calculated, now you can use the switches to input the banknotes. After selecting a type of banknote, press the insert button to confirm it. After confirming, the remaining payment will be displayed. Continue doing this until the display shows 0, or press next and go into the VERIFY state manually.



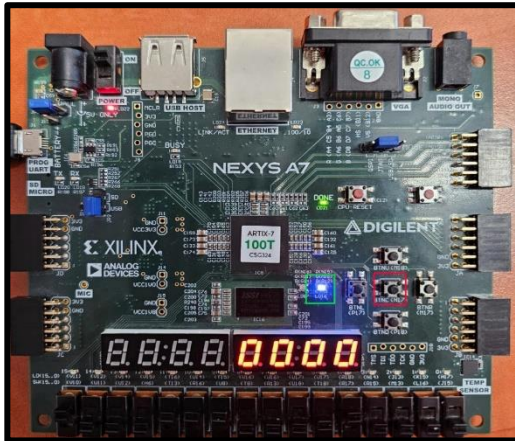
- In the VERIFY state, you will have to press the validation button to confirm all the input.



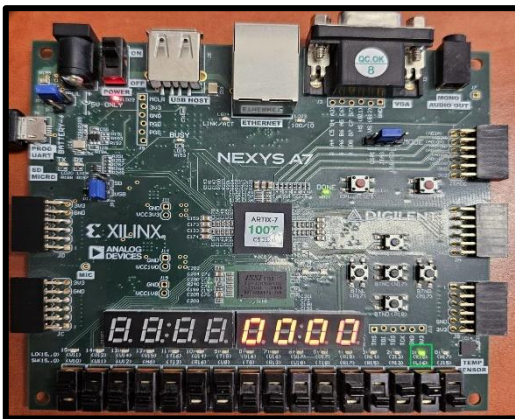
- Now, if you have inputted the correct amount of money, the ticket can be dispensed, and the change can be given, the machine will signal it with a green LED light and two other LEDs for the ticket and money, also the 7-segment displays the change. Press next to revert to the IDLE state.



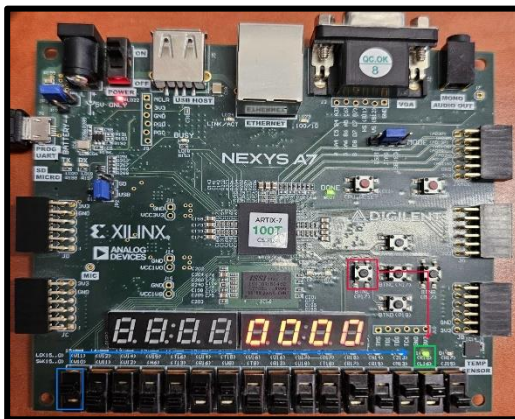
- However, if there are no more tickets available or the amount of money inserted was not enough, this will be signaled by a red LED. Press next to have your money refunded.



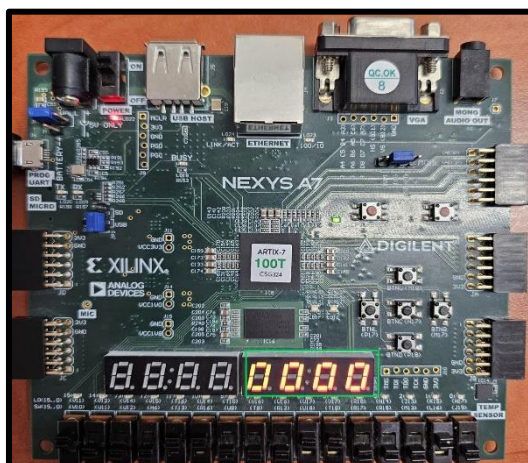
- In addition, if the paid amount was correct and there are enough tickets, but the machine is unable to give change, it will signal this with a blue LED. Press next to get your money refunded.



- After pressing next in the last 2 scenarios, the money will be refunded and signaled by an LED. Press next to revert to the IDLE state.



- Nevertheless, all actions can be canceled, except when the change is already being dispensed, by pressing the cancel button. Or flip the reset switch back to 0 to reset all the internal information in the machine to get a clean run the next time. Both of these actions trigger the REFUND state, and the refund will be signaled by an LED just like before.



- After finishing all actions, you will be back in the IDLE state, ready to start again.



## **5. FURTHER DEVELOPMENT AND IMPROVEMENTS**

### **5.1. ADVANCED PAYMENT AND BILLING SYSTEM**

- **NON-CASH TRANSACTIONS:** Currently, the system is designed for banknote-only processing. Future iterations could integrate credit card or NFC payment modules. This can be implemented as a separate “Payment Gateway” unit within the execution data path, using variation points in the source code to toggle between cash and digital payment modes.
- **DYNAMIC AUTO-BILLING:** The system can be expanded to support “Auto-billing” features where the machine calculates taxes, discounts, or loyalty points based on user profiles
- **SUBSCRIPTIONS:** Users can pay for a subscription for their usual destination, loaded with a certain number of trips. Instead of playing manually at the machine, they can scan the subscription to use up one travel.

### **5.2. USER INTERFACE ENHANCEMENTS**

- **HIGH-DEFINITION SCREEN INTERFACE:** Moving beyond the 4-digit 7-segment display and LED signals, an HDMI controller could be added to provide a Graphical User Interface. This would allow for displaying maps, multi-language instructions, error messages, and high-resolution price tables.
- **TOUCHSCREEN INPUT:** Replacing physical switches with a touchscreen interface using a communication protocol would significantly modernize the user interaction.

### **5.3. SYSTEM OPTIMISATION**

- **HIGH\_LEVEL SYNTHESIS:** To manage increasing complexity, portions of the VHDL code could be refactored using High-Level Synthesis. This approach allows for easier memory synchronization and performance scaling on modern FPGA platforms.
- **POWER AND TIMING ANALYSIS:** A critical next step for industrial deployment is performing a comprehensive power consumption analysis and timing closure to ensure the machine remains energy-efficient and responsive under all operating conditions.

### **5.4. MAINTENANCE MONITORING**

- **REMOTE INVENTORY TRACKING:** By adding a Wi-Fi or Ethernet module, the machine could report ticket and banknote levels to a maintenance service in real-time, enabling predictive maintenance, ensuring technicians refill the tickets or empty the vault before it runs out or becomes full.

## 6. BIBLIOGRAPHY

### 6.1. REFERENCES

Roth, C. H., & John, L. K. (2017). *Digital Systems Design Using VHDL* (3rd ed.). Cengage Learning:  
[http://www.r-5.org/files/books/computers/hw-layers/hardware/vhdl/Charles\\_H\\_Roth\\_Lizy\\_K\\_John-Digital\\_Systems\\_Design\\_Using\\_VHDL-EN.pdf](http://www.r-5.org/files/books/computers/hw-layers/hardware/vhdl/Charles_H_Roth_Lizy_K_John-Digital_Systems_Design_Using_VHDL-EN.pdf)

### 6.2. WEB AND VIDEO RESOURCES

Neso Academy – Finite State Machines (FSM):  
<https://www.youtube.com/playlist?list=PLLOaAE6VrtIRu6VI7gKVd9fw9ONRL3LJx>

LukeXII - [VHDL] : 7 Segment Display Decoder:  
<https://www.youtube.com/watch?v=YI3r3D-z658&t=393s>

IB Electronics World – 7 segment display on Basys 3(VHDL):  
<https://www.youtube.com/watch?v=gUiX8aMV5oE&list=WL&index=6&t=535s>

Xilinx Vivado Design Suite Documentation:  
<https://docs.amd.com/r/en-US/ug949-vivado-design-methodology/Maximizing-Impact-Early-in-the-Development-Cycle>